

BACCALAURÉAT

SESSION 2024

Épreuve de l'enseignement de spécialité

NUMÉRIQUE et SCIENCES INFORMATIQUES

Partie pratique

Classe Terminale de la voie générale

Sujet n°03

DURÉE DE L'ÉPREUVE : 1 heure

Le sujet comporte 4 pages numérotées de 1 / 4 à 4 / 4
Dès que le sujet vous est remis, assurez-vous qu'il est complet.

Le candidat doit traiter les 2 exercices.

EXERCICE 1 (10 points)

Écrire la fonction `maximum_tableau`, prenant en paramètre un tableau non vide de nombres `tab` (de type `list`) et renvoyant le plus grand élément de ce tableau.

Exemples :

```
>>> maximum_tableau([98, 12, 104, 23, 131, 9])
131
>>> maximum_tableau([-27, 24, -3, 15])
24
```

EXERCICE 2 (10 points)

On dispose de chaînes de caractères contenant uniquement des parenthèses ouvrantes et fermantes.

Un parenthésage est correct si :

- le nombre de parenthèses ouvrantes de la chaîne est égal au nombre de parenthèses fermantes ;
- en parcourant la chaîne de gauche à droite, le nombre de parenthèses déjà ouvertes doit être, à tout moment, supérieur ou égal au nombre de parenthèses déjà fermées.

Ainsi, `((()())(()) )` est un parenthésage correct.

Les parenthésages `()))( )` et `((() )()` sont, eux, incorrects.

On dispose du code de la classe `Pile` suivant :

```
class Pile:
    """Classe définissant une structure de pile."""
    def __init__(self):
        self.contenu = []

    def est_vide(self):
        """Renvoie un booléen indiquant si la pile est vide."""
        return self.contenu == []

    def empiler(self, v):
        """Place l'élément v au sommet de la pile"""
        self.contenu.append(v)

    def depiler(self):
        """
        Retire et renvoie l'élément placé au sommet de la pile,
        si la pile n'est pas vide. Produit une erreur sinon.
        """
        assert not self.est_vide()
        return self.contenu.pop()
```

On souhaite programmer une fonction `bon_parenthesage` qui prend en paramètre une chaîne de caractères `ch` formée de parenthèses et renvoie `True` si la chaîne est bien parenthésée et `False` sinon.

Cette fonction utilise une pile et suit le principe suivant : en parcourant la chaîne de gauche à droite, si on trouve une parenthèse ouvrante, on l'empile au sommet de la pile et si on trouve une parenthèse fermante, on dépile (si possible) la parenthèse ouvrante stockée au sommet de la pile.

La chaîne est alors bien parenthésée si, à la fin du parcours, la pile est vide.

Elle est, par contre, mal parenthésée :

- si dans le parcours, on trouve une parenthèse fermante, alors que la pile est vide ;
- ou si, à la fin du parcours, la pile n'est pas vide.

Compléter le code de la fonction `bon_parenthesage` ci-dessous:

```
def bon_parenthesage(ch):  
    """Renvoie un booléen indiquant si la chaîne ch  
    est bien parenthésée"""  
    p = Pile()  
    for c in ch:  
        if c == ...:  
            p.empiler(c)  
        elif c == ...:  
            if p.est_vide():  
                ...  
            else:  
                ...  
    return ...
```

Exemples :

```
>>> bon_parenthesage("((()())(()))")  
True  
>>> bon_parenthesage("() ) (())")  
False  
>>> bon_parenthesage("(()) (())")  
False
```